Pipeline e Execução Especulativa

Professora Quelita Ribeiro

MIPS

- Exemplo didático em livros
- Stanford MIPS comercializado por MIPS Technologies
- Fatia do mercado de núcleos embarcados
- Aplicações em eletrônica de consumo, equipamentos de rede, armazenamento, impressoras…
- Típico de conjunto de instruções modernos
- Características RISC

Operações Aritméticas: MIPS

Todas operações aritméticas tem a mesma forma (simples e regular)

Três operandos: duas origens e um destino

add a, b, c -> a = b + c

Registradores como operandos: MIPS

- Instruções aritméticas usam registradores como operandos
- MIPS tem um conjunto de 32 registradores de 32 bits
  - Numerados de 0 a 31
  - Dados de 32 bits são chamados *word*

Registradores: MIPS

- $t0, $t1, ..., $t9 -> valores temporários
- $s0, $s1, ..., $s7 -> valores a serem salvas
- Registrador 0 ($zero) do MIPS é a **constante** 0, não pode ser sobrescrita

EX: mover conteúdo de registradores

Add **$t2**, $s1, $zero

Resumo

Repertório de instruções

- ULA: add, and, sub, or…
- Transferência de dados: load, store
- Controle: bne, beq, j…

- Tipo e tamanho dos operandos (dados)
  - Byte, half word, word
- Formato de instrução
  - 32 bits cada instrução (palavra)
  - 3 tipos de instruções

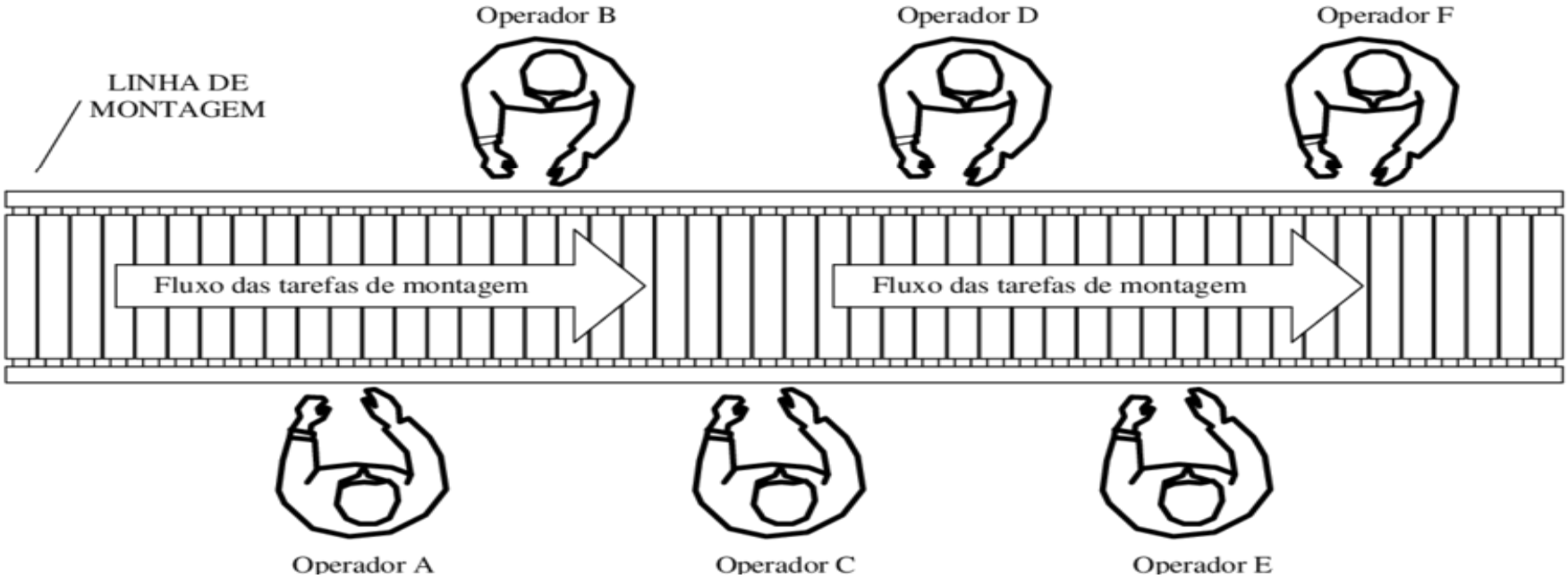
Pipeline

- Como melhorar o desempenho do processador?
  - Buscar forma de paralelismo!

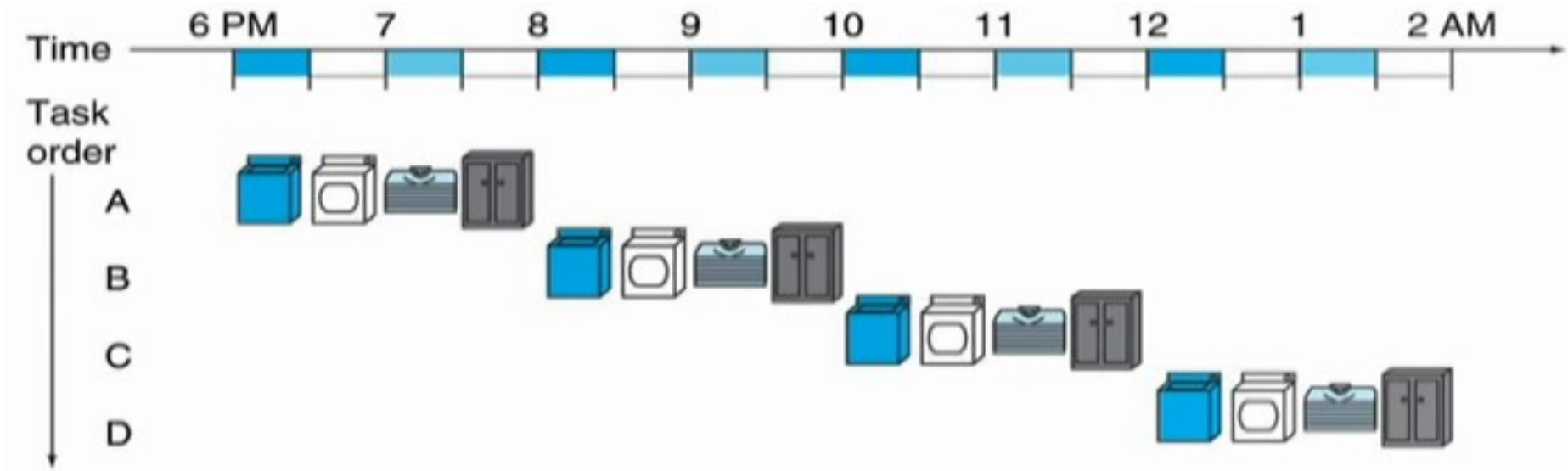
- Você vai entender como um único processador pode executar várias tarefas ao mesmo tempo

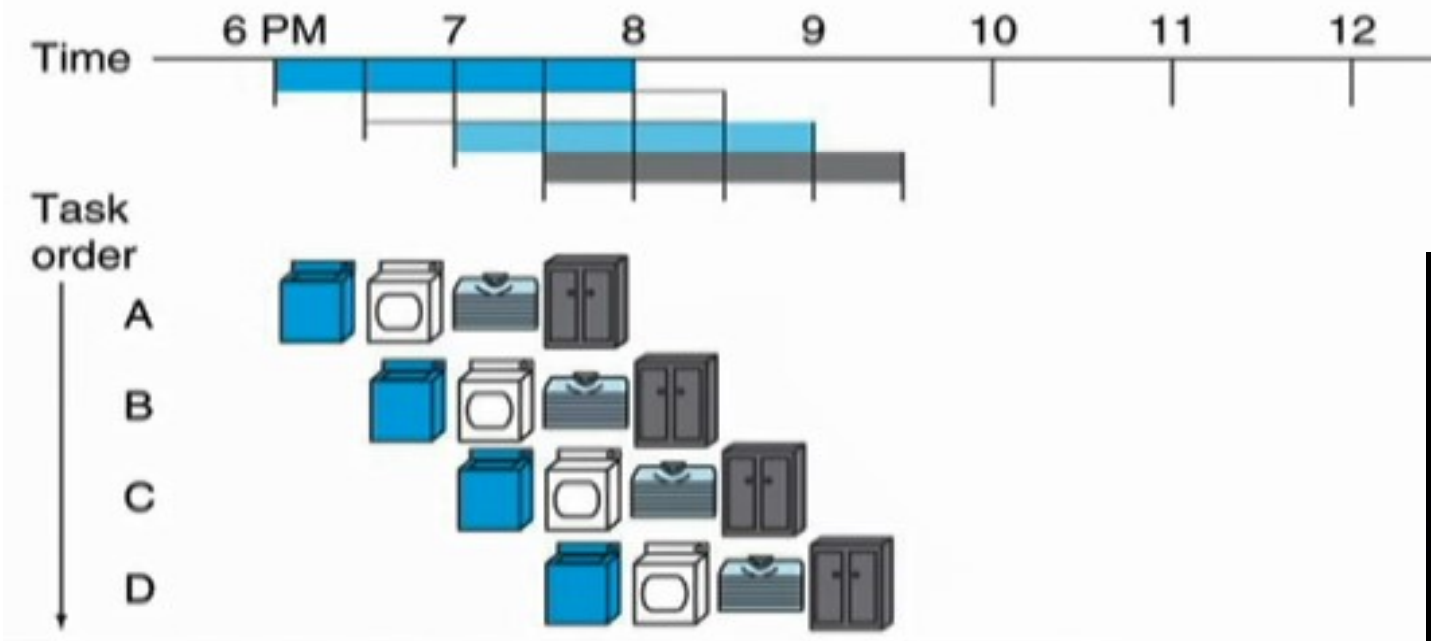
Estratégia de Pipeline

- O processador usa técnica de *Pipeline* de instruções
- Execução de instruções como uma "linha de montagem"



Pipeline





Operação contínua:
1º lote após 4 estágios, outros após 1 estágio!

Speedup = 8/3,5 = 2,3
Speedup ~ número de estágios

MIPS Pipeline

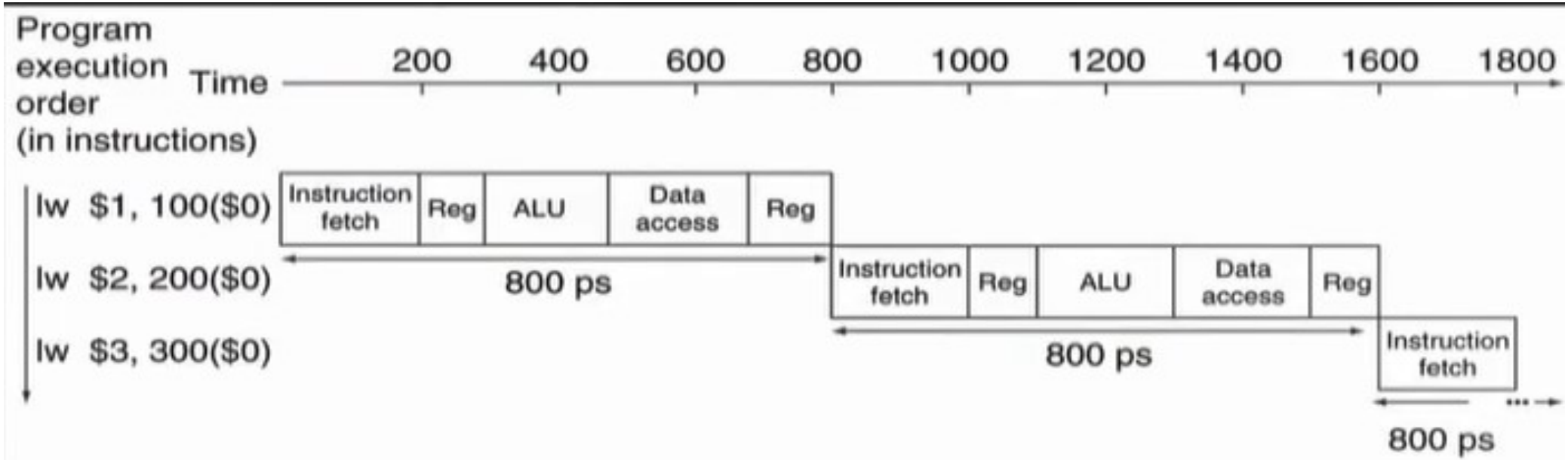
- Envolve dividir o ciclo de instrução em um determinado número de estágios:
  - **IF (Instruction Fetch): Busca de instrução na memória**
  - **ID (Instruction Decode): Decodificação de instrução e leitura de registradores/operandos**
  - **EX : Executa operação (associada a operações lógicas e aritméticas) ou calcula endereço de operandos**
  - **MEM: acessa o operando na memória**
  - **WB: escreve o resultado no registrador**

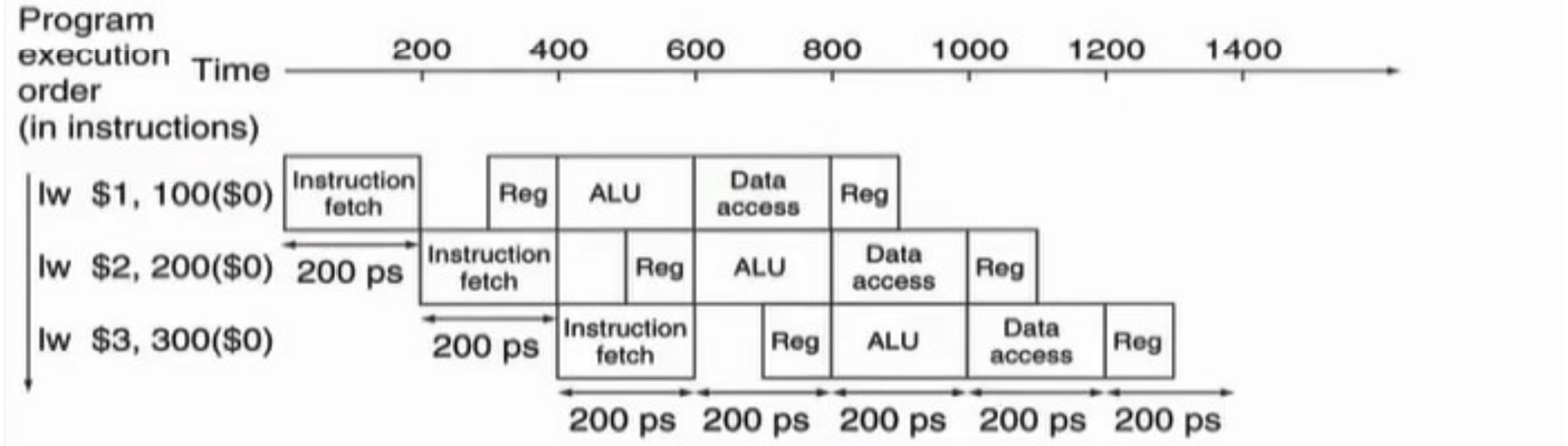
MIPS ciclo único

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

Maior atraso é a instrução load, que determina o período do clock -> Tc = 800 ps

MIPS Pipeline





Hazards

Situações que impedem o inicio da próxima instrução no próximo ciclo:

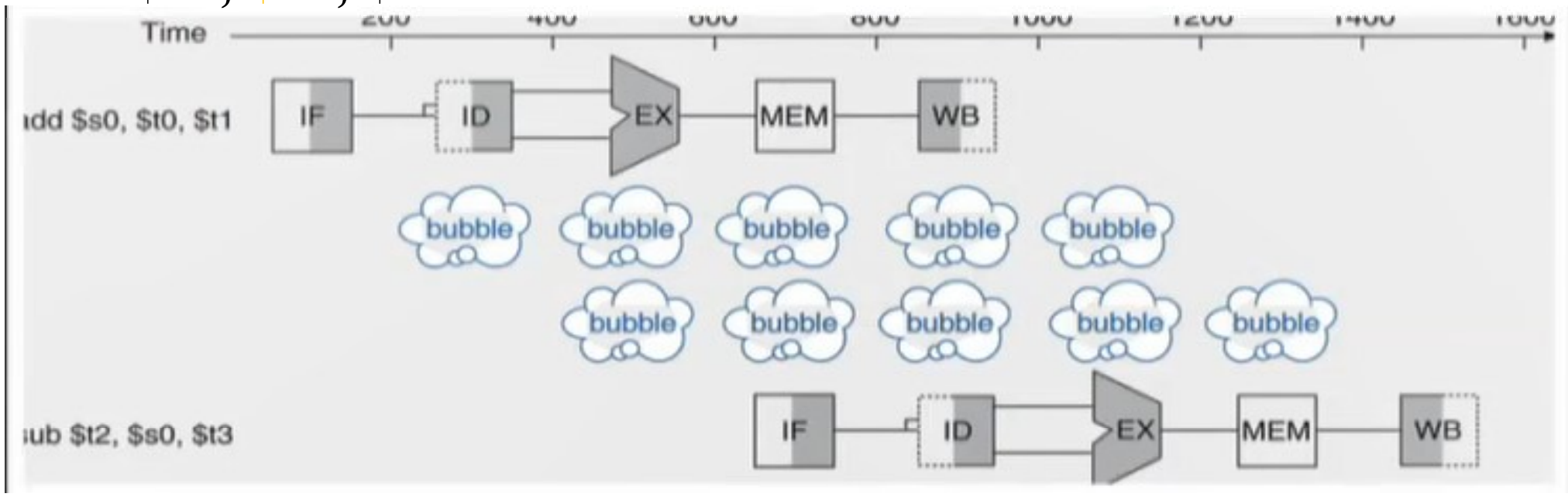
- Conflitos estruturais (*Structure hazards*)
  - Um recurso necessário está ocupado

    Ex: ler o registrador que está sendo escrito por outra instrução
- Conflito de dados (*Data hazards*)
  - Instrução anterior completa leitura/escrita de dados
- Conflito de controle (*Control hazard*)
  - Decisão de controle depende de instrução prévia

Data Hazards

add $s0, $t0, $t1

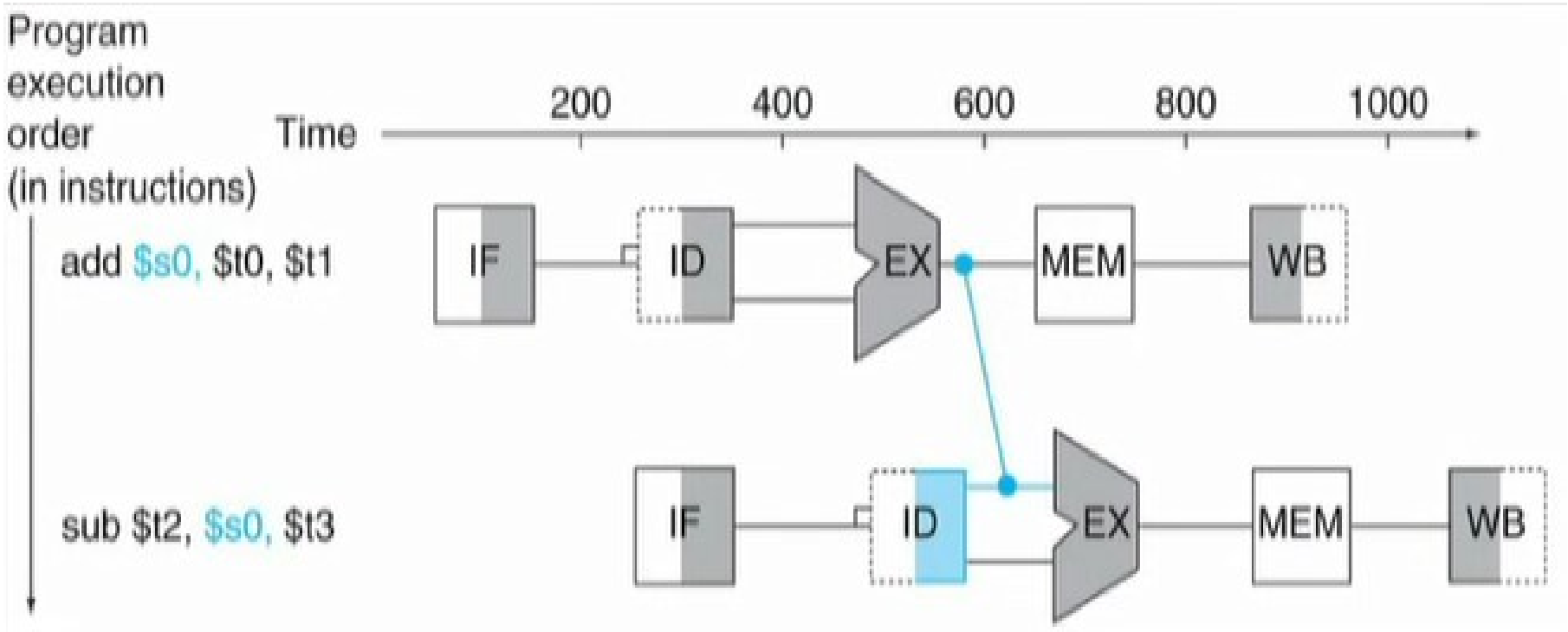
sub $t2, $s0, $t03



- O resultado de $s0 será escrito no registrador no 5° estágio.
- A instrução sub precisa ler o resultado no 2º estágio
- Perder dois ciclos de execução do pipeline

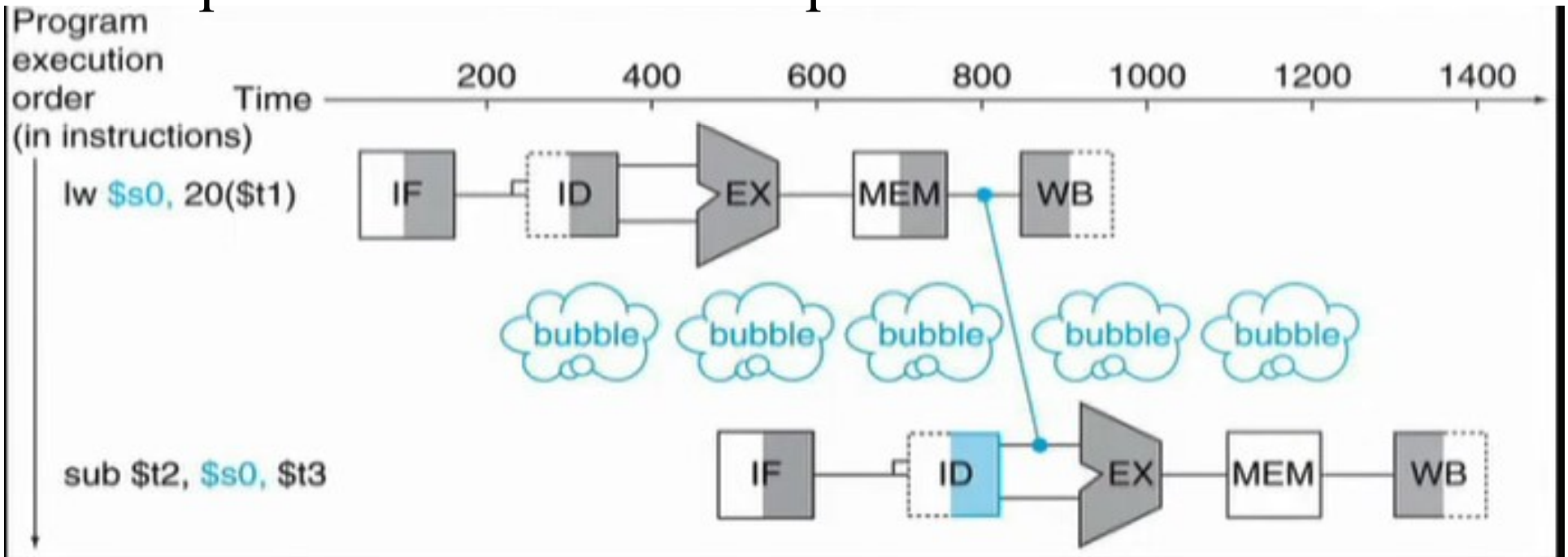
Encaminhamento (forwarding)

- Utiliza o resultado quando é calculado
- Não espera pelo armazenamento no registrador
- Requer conexões extras no fluxo de dados



Encaminhamento com load

- Nem sempre é possível evitar a parada…
  - O dado só estará pronto no final do estágio no acesso de dados na memória
- Se o valor não estiver disponível quando necessário, não é possível voltar no tempo

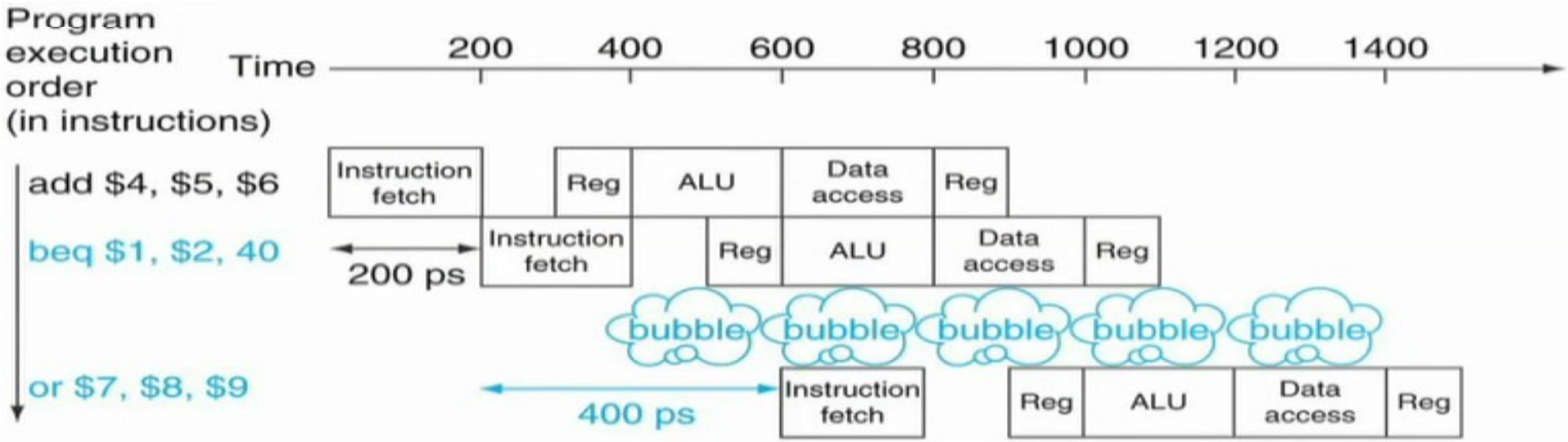


Control Hazards

- Desvios (branch) determina fluxo de controle
  - A instrução de desvio vai ser executada? Só sabemos no estagio de execução (depois da comparação e do cálculo de endereço)
  - Várias bolhas no pipeline
  - Busca da próxima instrução depende do resultado do desvio
- MIPS pipeline:
  - 1° estratégia : tentar adiantar a resolução do desvio, o mais cedo: seria o estágio ID e assim esperar somente 1 ciclo

Resolução do desvio em ID

- Fazer a busca da próxima instrução correta
  - Inserir uma bolha no pipeline, no caso do MIPS isso não é tão ruim…
  - Processadores mais modernos tem mais estágios e dificilmente conseguem trazer essa informação tão cedo



Execução Especulativa

Execução Especulativa

- Para que o potencial de desempenho oferecido pelos processadores com pipeline seja explorado, é necessário que **não haja paralisação do processador** por conta de **dependências no fluxo de execução**.
- Tem como objetivo executar instruções antes que suas dependências sejam totalmente resolvidas.

Previsão de desvio

Várias técnicas podem ser usadas para prever se um desvio será tomado ou não. Entre as mais comuns estão as seguintes:

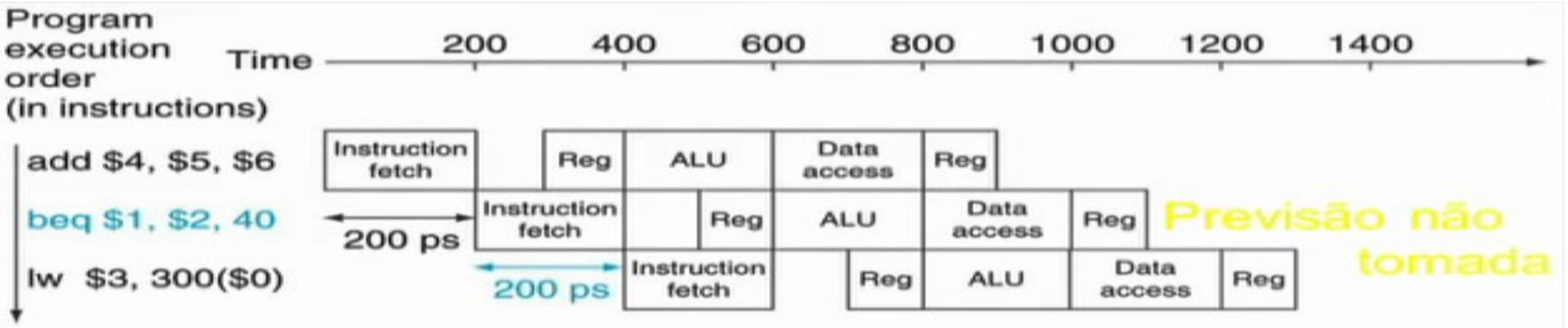
- Prever que o desvio nunca será tomado
- Prever que o desvio sempre será tomado
- Prever se o desvio será tomado ou não conforme o código de operação
- Prever o desvio com base em uma tabela de histórico de desvios

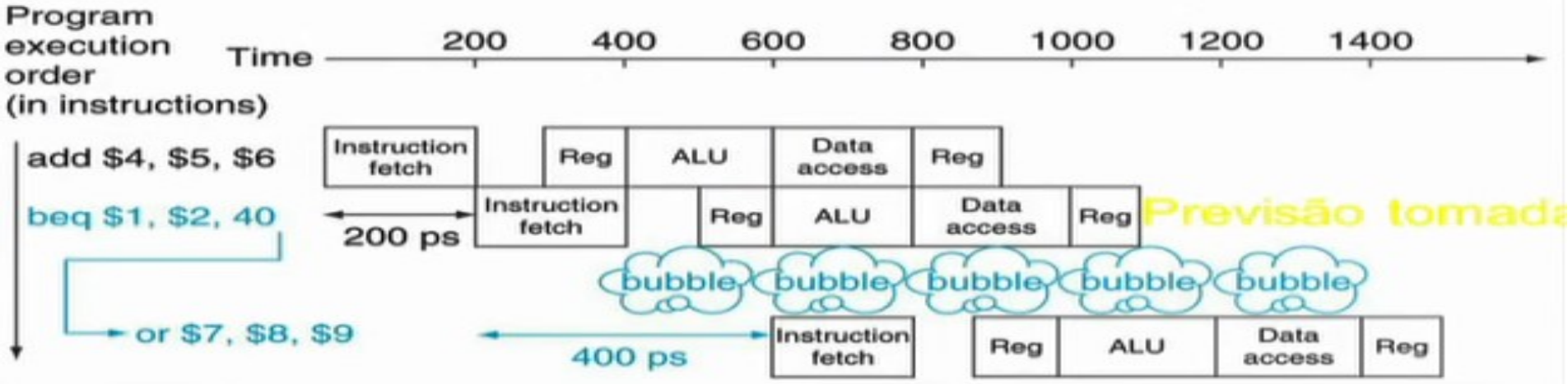
Previsão de desvios no MIPS

- Pipelines mais longos não conseguem determinar resultado do desvio mais cedo
- Inserção de bolhas gera muito atraso
- MIPS pipeline:
  - Solução: prever resultado do desvio
  - Previsão que o desvio nunca será executado
    - Vai me dar uma sequência de instruções
  - Parada somente se previsão estiver errada
  - Busca instrução depois do branch, sem atrasos.

Previsão de desvios no MIPS

- Transforma aquela execução errada numa bolha





Exercícios

1. Defina Execução especulativa e as técnicas existentes?
2. Defina pipeline e sua importância no desempenho dos processadores.
3. Enumere e dê exemplo dos tipos de conflitos que podem ocorrer em um pipeline de instruções de um processador.

Referências Bibliográficas

- HENNESSY, John L.; PATTERSON, David A. **Organização e projeto de computadores: a interface hardware/software**. Elsevier Brasil, 2014.
- STALLINGS, William. Arquitetura e Organização de Computadores. 8° Edição. 2010.
- MONEY HARRIS, David; HARRIS, Sarah L. Digital design and computer architecture. 2° Edição, Elsevier. 2013.